TinyTracer Host: Design Doc

Summary

This task builds tinyTracerHost, the laptop-side tool that compiles a scene into TinyTracer SRAM words, drives a render over UART, and turns the returned pixels into a finished image. The chip returns linear 8-bit pixels, averaged over at most 32 samples and clamped at 255, so gamma correction and noise removal have to happen on the host.

The proposal has four deliverables:

1. **Core host script.** Scene file, scene compiler, UART transport, pixel receiver, PNG output. One command renders a scene.
2. **Image pipeline.** Multi-pass accumulation, firefly clamp, edge-aware denoise, sRGB gamma.
3. **Viewport scene designer.** A real-time raycast or rasterizer preview that uses the chip's camera model, with an editor and a "send to chip" button.
4. **Logging and Debugger.** A log file to show all the debug information of the UART values coming in from the TinyTracer.
5. **Artifact study.** A harness that compares the chip against the software model and a floating-point reference, and a catalogue of what differs and why.

** Maybe able to add 3D render studies like Utah Teapot or Stanford Dragon for the scene designer.

Background + Overview

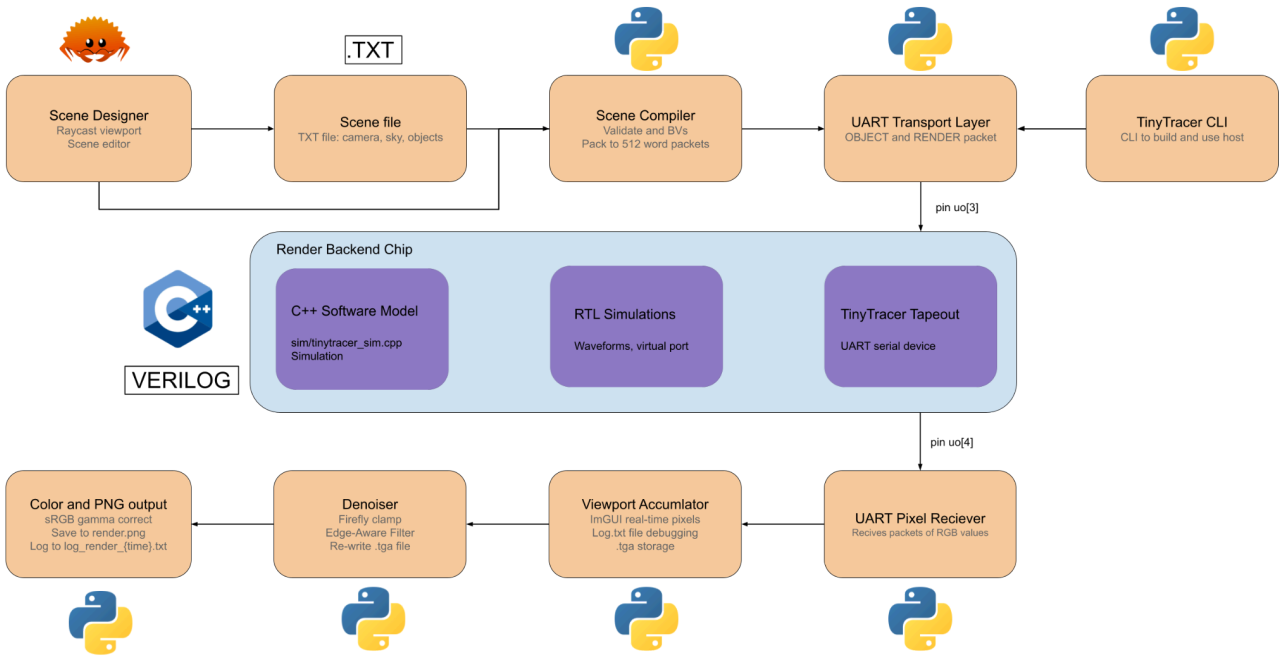
TinyTracer renders one pixel at a time and the host does everything else: it builds the scene, computes the camera vectors and bounding volumes, and receives the pixels. The facts below come from the TinyTracer documentation and the team's rendering-flow reference, and set the constraints for this design.

All the constraints given are from the renderflow and the docs linked above. All the external host scripts run independently and communicate to the python scripts via the .txt scene scripts. The python script compiles the scene.txt to 512 word packets to be sent over UART and the receiver will collect all the incoming pixels to show real-time and manipulate (denoise and gamma correct). The scene designer is completely separate from the existing python core script.

Scene Designer:

- Abstract object placement
- Easier to make scenes the better

Architecture



System architecture of the host python script, may add c++ or rust simulations.

Tech Stack

Python scripts are used for the whole host. ImGUI library is used for the viewport for the incoming pixel information, and a potential scene creator. The host handles 262,144 pixels (512 x 512) per frame sending over scene data to the TinyTracer chip to be rendered. Viewport will use simple raycasting or rasterization pipeline to create a general scene before the film quality image is rendered.
Change to have a different language for the scene designer. (C++? Rust?) Rasterization or raycast pipeline

Layer	Choice	Why
Language	Python 3.11 or newer	Team can read and extend it; `tomllib`, dataclasses and typing in the standard library
Serial link	`pyserial`	Standard for UART from a laptop; works with USB-serial adapters on Linux, Windows, macOS
Image maths	`numpy`	Accumulation, gamma, filters as array operations
Filters and metrics	`opencv-python`, `scikit-image`	Bilateral and non-local-means filters; PSNR and SSIM
Image output	`Pillow` for PNG, `imageio` for GIF and MP4	PNG text chunks hold the render metadata
Viewport rendering	`moderngl` with a GLSL fragment shader	Raycasts spheres and triangles per pixel; 70 objects is trivial for any GPU
Viewport UI	`imgui-bundle` (Dear ImGui)	Object list, sliders, colour pickers, budget meter with little code
CLI	`argparse` or `typer`	`render`, `compile`, `view`, `study` subcommands
Software model	`sim/tinytracer_sim.cpp` , run as a subprocess	Bit-accurate 16-bit arithmetic, already written by the team
RTL simulation	Verilator or cocotb behind a pseudo-terminal	Same UART bytes as the chip, no hardware needed
Tests	`pytest`, `ruff`, GitHub Actions	Encoding round-trips and golden images on every push

Packaging	`pyproject.toml`, `uv` or `pip`	`pip install -e .` gives the `tthost` command

Scene Format and Chip Link

A scene file comes in two interchangeable formats, text (.txt) and JSON (.json); the compiler turns either into the 512-word SRAM image defined in Scene Encoding. The file is what the designer saves, what the simulator reads and what gets stored beside every render.

Link to .txt demo

Line	Fields	Notes
`image`	Size	512; the chip draws 512 × 512 only
`spp`	Samples per pixel	1, 2, 4, 8, 16 or 32
`camera`	Position x y z, yaw from +x, pitch, field of view, in degrees	The host moves the scene so the camera sits at x = 0, y = 0
`sky`	Horizon colour, top colour	0 to 255 per channel
`ground`, `checker`	Checker colours A and B	Ground is fixed at z = 0
`sphere`	Centre x y z, radius	Then material, colour, optional strength and bounding volume
`tri`	Three corners	Same trailing fields
`quad`	Four corners in order	Host makes 2 triangles
`box`	Two opposite corners	Host makes 12 triangles

Materials are `matte`, `mirror`, `glass` (spheres only) and `glow` (strength 0 to 15.99). The trailing number assigns the object to a bounding volume, 0 to 15. The demo scene has 6 spheres and 18 triangles in 4 bounding volumes, and compiles to 295 words.

Scene compiler

1. **Validate** against the chip's host rules: positions within ±256, objects at z ≥ 0, field of view at most 90°, no glass triangles, samples a power of 2 up to 32. Errors name the line and the rule. Quad and box lines are expanded first, into 2 and 12 triangles.

2. **Translate** the scene so the camera sits at x = 0, y = 0; the camera height becomes `CAM_Z`.
3. **Compute camera vectors** from yaw, pitch and field of view: `F` with length 1, `R` and `U` with length tan(fov / 2).
4. **Build bounding volumes.** Group objects by the bounding-volume number on each line. Objects with no number are clustered automatically (k-means on centres is enough at this scale). Then fit a bounding sphere to each group. Every object must lie fully inside its sphere: the chip assumes a bounce starts inside its own volume.
5. **Lay out memory.** Header at 0 to 16, bounding volumes from 17, primitives after them, grouped by bounding volume, with `BV_START` and `BV_COUNT` filled in. Fail if the total passes 512 words.
6. **Quantize** to Q9.7, Q2.14 and Q4.10, and warn on any value that saturates.
7. **Emit** the word list, plus a readable hex dump for debugging against the RTL.

A decoder that reverses steps 5 to 7 ships with the compiler. Encode then decode must return the same scene to within one quantization step; that is the main unit test.

UART transport

Frames follow UART Frame Encoding.

Message	Direction	Bytes	Escaped data bytes
OBJECT	Host to chip	`0x01`, word low, word high, address low, address high	`0x00`, `0x01`, `0x03`
RENDER	Host to chip	`0x00`, width low, width high, height low, height high	`0x00`, `0x01`, `0x03`
PIXEL	Chip to host	`0x00`, red, green, blue	`0x00`, `0x03`

- An escape is the byte `0x03` sent before the data byte.
- A full scene is at most 512 OBJECT messages, about 2,560 bytes before escapes: roughly 0.2 s at 115,200 baud. The demo scene is 295 words, about 0.13 s. Upload time is negligible.
- **Delta upload.** The host keeps the last image it sent and writes only changed words. Turning the camera or changing its height changes up to 10 words. Moving it sideways shifts every object, because the chip's camera is fixed at x = 0, y = 0.

Pixel receiver

PIXEL messages carry no coordinates, so the receiver's job is to never lose its place.

- A reader thread drains the serial port into a queue; a state machine unescapes bytes and assembles pixels.
- Data bytes equal to `0x00` are always escaped, so a bare `0x00` is always the start of a pixel. If a pixel arrives short, it is marked bad and the receiver resyncs on the next start byte.
- The render ends at 262,144 pixels (512 × 512) or on a timeout. A short count is reported as an error with the image saved as far as it got; bad pixels are filled from neighbours and listed in the metadata.
- Each pixel is timestamped on arrival. That gives a live progress bar, an ETA, and the cost heat map described under other features.

Live render view

The team asked to watch the pixels come in, so the core tool opens a window that fills in as the chip renders, and the finished frame is compiled into an image afterwards.

- **What it shows.** A 512 × 512 frame that starts empty and fills row by row, top to bottom, in the order the chip sends pixels. At 115,200 baud a row takes at least 0.18 s, so the window redraws once per completed row and whenever 50 ms pass.
- **How it draws.** The receiver writes each pixel into the linear frame buffer. The window uploads changed rows to a GPU texture and applies sRGB gamma in the display shader, so the stored data stays linear and untouched.
- **What it reports.** Pixels received, current row, elapsed time, estimated time left, pixel rate, and a count of bad or resynced pixels. A marker line shows the row being rendered.
- **Across passes.** With several passes, the window shows the running average, so the image visibly cleans up pass by pass.
- **Compile to image.** When the last pixel arrives, or when you stop early, the frame goes through the image pipeline and is saved as a PNG with its metadata, plus the raw linear buffer. A key toggles the view between raw and processed.
- **Without a display.** `--no-window` runs the same render with a text progress bar, for scripts, CI and overnight batches.

The window uses the same `moderngl` and Dear ImGui stack as the scene designer. The designer later embeds this view beside its preview, so nothing is built twice.

Render log

Every render writes two log files beside its image, flushed to disk as data arrives, so an interrupted run keeps everything received up to that point. No separate checkpoint system is needed. The chip cannot resume mid-frame, so an interrupted render restarts from the first pixel.

File	Contents	Used for
Raw byte log	Every byte received and sent, each with a timestamp, before any unescaping	Ground truth when something looks wrong; input to the replay backend
Event log	One JSON object per line: what the host did and saw	Reading by eye, loading into NumPy, the cost heat map

The event log records:

- **Run header.** Date, scene file name and hash, the SRAM words sent, samples per pixel, baud rate, serial port, backend, and the host's git commit.
- **Sent messages.** Each OBJECT message and the RENDER message with its timestamp, which marks the start of the render.
- **Each pixel.** Index, x, y, red, green, blue, arrival time, and whether any byte was escaped.
- **Anomalies.** Short pixels, resyncs, unexpected bytes, gaps longer than a set threshold, and timeouts.
- **Summaries.** Completion time of each row; at the end, pixels received against the 262,144 expected, total time, average pixel rate and the count of bad pixels.

A full frame of per-pixel lines is roughly 10 MB as text.

Replay backend. A fourth backend reads a raw byte log and plays it back as if it were the chip, at the original timing or as fast as possible. A fault seen on hardware can then be reproduced exactly, and the live view can be demonstrated with no chip present.

Image pipeline

Yes, gamma correction belongs here, as the last step; and denoising happens before it, on linear values. The host only ever sees finished pixels, never individual samples, which decides where each denoising idea can live.

Stages, in order

1. **Accumulate.** Render the scene N times and average the passes per pixel in 32-bit floats.
2. **Reject outliers.** Across passes, drop values far from the per-pixel median. Within a pass, clamp a pixel that is far brighter than the median of its 3 × 3 neighbours.
3. **Denoise.** Edge-aware spatial filter, guided by noise-free buffers from the host's own raycaster.
4. **Expose.** Optional multiply; a no-op by default.
5. **Encode.** Linear to sRGB with the standard transfer function, then round to 8 bits with optional dithering.
6. **Save.** PNG with the scene, settings and timings in its metadata, plus the raw linear buffer as `.npy`.

Denoiser options

Build them in this order; each is a function from a linear image to a linear image, so they are swappable and measurable.

Filter	How it works	Strength	Weakness
Gaussian	Weighted average of neighbours	5 lines of code	Blurs edges and the checkerboard
Median 3 × 3	Middle value of neighbours	Removes isolated fireflies	Softens fine detail
Bilateral	Neighbours weighted by distance and colour similarity	Keeps strong edges	Noise can pass as an edge at low sample counts
Non-local means	Averages similar patches from across the image	Good on flat regions and repeated texture	Slow, can smear
Guided À-Trous	Bilateral weights taken from noise-free albedo, normal and depth	Sharp edges at 8 samples per pixel	Needs the host raycaster; reflections and glass are not in the guides

The guided filter is the one worth proposing. The host knows the scene exactly, so the viewport's raycaster can produce albedo, surface normal and depth for every pixel with no noise. Those buffers tell the filter where real edges are. Dividing the image by albedo before filtering and multiplying back afterwards keeps the checkerboard and object colours crisp while the lighting is smoothed.

Colour convention

The pipeline treats scene colours as linear values, so the designer's colour pickers convert from sRGB before packing. This needs agreement with the team; it is listed under open questions.

Viewport Scene Designer

The designer is a desktop window where you build and frame a scene in real time, then press one button to have the chip render it. Today a scene is a hand-written list of numbers and each test of a camera angle costs minutes of chip time; the viewport makes that loop instant.

Will use Rust or C++ to build that can compile into a .txt file that can be sent over UART through the Python core script.

How it renders

A full-screen GLSL fragment shader casts one ray per pixel against the scene's spheres, triangles and ground plane. Three properties make this the right approach rather than a rasterizer:

- **Same camera model.** The shader builds its rays from the same `F`, `R`, `U` vectors the compiler sends to the chip, so framing matches the chip render.
- **Exact spheres.** Ray-sphere intersection is analytic; there is no mesh approximation to disagree with the chip.
- **Same scene data.** Objects are uploaded as a small uniform array straight from the scene file. At 70 objects or fewer, a loop per pixel runs at full frame rate.

Shading is deliberately simple: surface colour with a fixed light for shape, the two-colour sky gradient, the 4 × 4 checker ground, a tint for mirrors and glass, and a bright fill for emissive objects. Triangles are shaded on both sides and the ground stops at distance 256, as on the chip. The frame is always square, since the chip draws 512 × 512 only. The preview shows layout and framing; the chip supplies the real lighting.

What the editor does

- Fly camera with mouse and WASD, with field of view capped at 90°.
- Add, select, move and delete spheres and triangles; edit material, colour and glow strength.

- **Memory meter**: words used out of 512, updated on every edit, with the bounding-volume split shown.
- **Live validation**: objects below the ground, positions out of range and glass triangles are flagged as you edit.
- View modes: shaded, albedo, normals, depth, and bounding volumes as wireframe spheres.
- Save and load the scene file.
- **Render on chip**: compiles, uploads, and fills the chip's image in beside the preview row by row as pixels arrive.

Glossary

- **Noise** is random error. Each sample follows one random path, so the samples of a pixel disagree and their average is only an estimate. It shrinks with more samples: 4 times the samples halves it.
- **An artifact** is systematic error. It stays the same or forms a pattern however many samples are taken: banding from fixed-point rounding, a clipped highlight, a shifted row from a lost UART byte.
- **Gamma** is not the brightness of the scene. It is the non-linear encoding that displays and image files expect. The chip outputs values proportional to light; shown directly they look too dark. Gamma correction converts linear values to sRGB as the last step.